

Anteater Chess

Version 0.3

Software Architecture Specification

EECS 22L: Software Engineering Project in C
University of California, Irvine
Spring 2026

Team 22: DeepAnteater

Yao Li
Benjamin Feng
Yide Li
Yurang Li
Yasith Diunugala
Max Zhang

Table of Contents

Table of Contents.....	2
Glossary.....	4
Software Architecture Overview.....	5
1.1. Major Software Components.....	6
1.1.1. Modular Hierarchy Overview.....	6
1.1.2. Root-Level Project Structure.....	7
1.1.3. Module Organization.....	9
1.2. Main Data Structures Overview.....	14
1.3. Overall Program Control Flow.....	17
1.3.1. System States Definition.....	17
1.3.2. Event Definition.....	18
1.3.3. Control Model Overview.....	19
1.3.4. Event-Driven Execution Model.....	20
1.3.5. Top-Level Control Flow.....	21
1.3.6. Gameplay Loop.....	21
1.3.7. Game Termination Flow.....	23
2. Installation.....	25
2.1. System Requirements and Compatibility.....	25
2.2. Setup and Configuration.....	25
3. Documentation of Packages, Modules, and Interfaces.....	25
3.1. Main Data Types and Structures.....	25
3.1.1. Board Representation.....	25
3.1.2. Piece Structure.....	26
3.1.3. Position.....	27
3.1.4. Move Representation.....	27
3.1.5. Command.....	29
3.1.6. Event.....	29
3.1.7. Event Queue.....	30
3.1.8. Player.....	32
3.1.9. System State.....	32
3.1.10. Game Configuration.....	33
3.1.11. Move List.....	34
3.1.12. Game State.....	35
3.1.13. Log System (File Layer).....	36
3.1.14. Timer System.....	37
3.1.15. Transposition Table Entry.....	38
3.2. Detailed Description of Functions and Parameters.....	39
3.2.1. Core Data Headers.....	39
3.2.2. Gameplay Logic Headers.....	55

3.2.3. Turn and Time Headers.....	61
3.2.4. Event and Control Headers.....	68
3.2.5. Input Headers.....	74
3.2.6. UI Headers.....	78
3.2.7. AI Headers.....	87
3.2.8. Services Headers.....	89
3.3. Detailed Description of Input and Output Formats.....	93
3.3.1. The syntax or format of menu selections by the user.....	94
3.3.2. The syntax or format of a move entered by the user.....	95
3.3.3. The syntax or format of the log file.....	97
4. Development Plan and Timeline.....	100
4.1. artition of tasks.....	100
4.2. Parallel Development Strategy.....	106
4.3. Development Timeline (Apr 12 – Apr 19).....	106
4.4. Team member responsibilities.....	107
5. Back Matter.....	109
5.1. Copyright.....	109
5.2. References.....	109
5.3. Index.....	109

Glossary

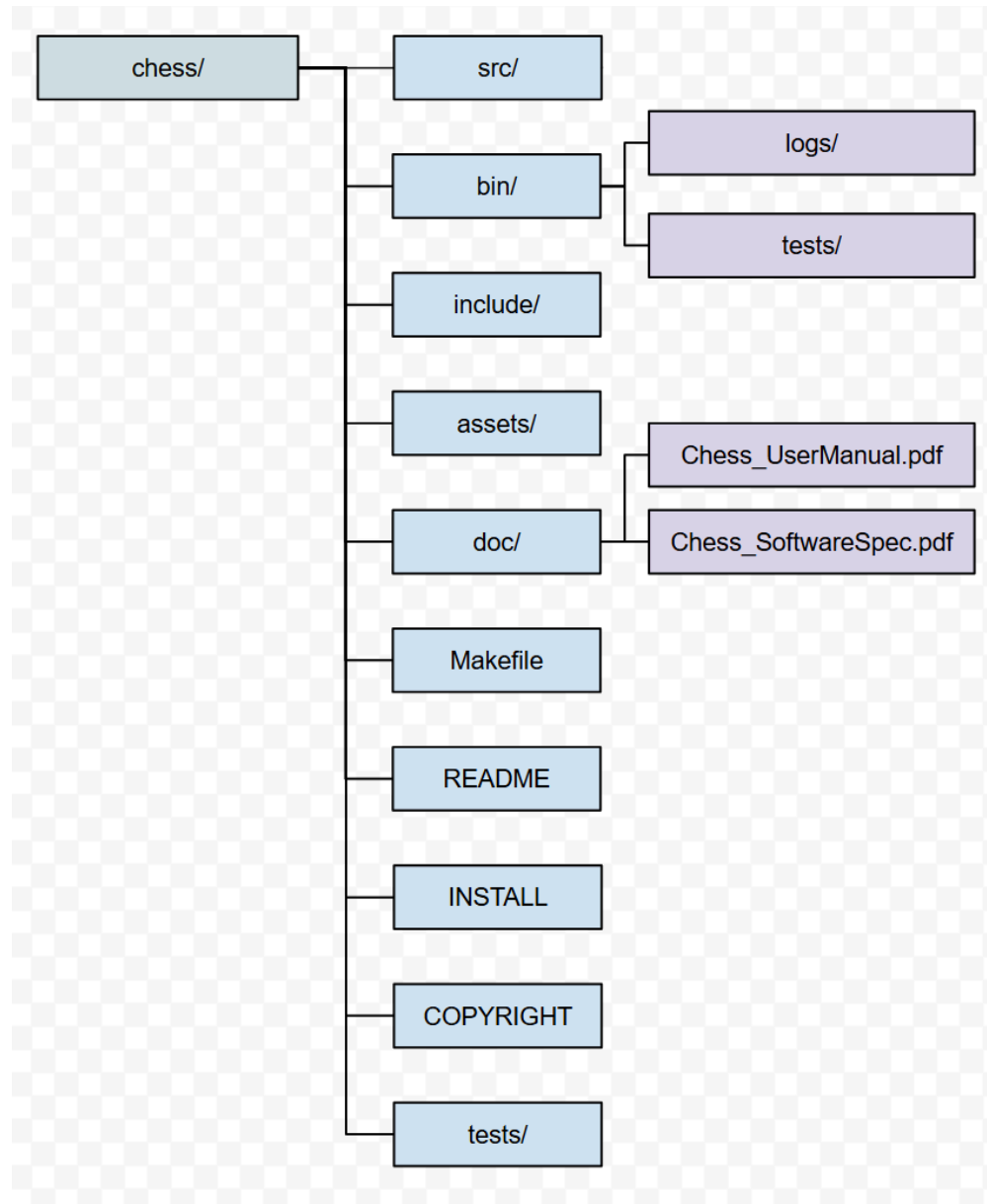
Term	Definition
Ant	Another name for a pawn in Anteater Chess.
Anteater	A new chess piece introduced in Anteater Chess. It moves one square in any direction (like a king) but can only capture pawns.
Board	A fixed-size 2D array representing the chessboard where each cell stores a Piece value.
Checkmate	A check from which the king cannot escape. The game ends, and the player whose king is in checkmate loses.
Command	Represents parsed user input, decoupling the UI input format from internal move representation.
Event	Represents a high-level system action processed by the FSM, unifying all system interactions under a single data type.
EventQueue	A multi-queue structure consisting of ControlQueue, SystemQueue, and GameplayQueue, used to store and prioritize events before they are processed by the FSM.
ControlQueue	A high-priority queue that stores control-related events such as menu actions, exit requests, and fatal errors.
SystemQueue	A medium-priority queue that stores system-generated events such as timer expiration.
GameplayQueue	A low-priority queue that stores gameplay-related events such as move input and AI actions.

FSM (Finite-State Machine)	Controls system behavior at a high level, determining how incoming events are processed and defining valid state transitions.
GameConfig	Encapsulates all user-defined configuration parameters selected during the game setup phase.
GameState	Represents the complete runtime state of an active game session, including board configuration, players, and progression.
Log	Represents the persistent record of gameplay actions generated during execution. Stores moves and related events in a human-readable format for replay, debugging, and post-game analysis.
Move	Represents a complete gameplay action performed by a single piece.
MoveList	Represents a generic container for storing a collection of Move objects.
Piece	Represents a single chess unit, combining its piece type and ownership color.
Position	Represents a coordinate on the board, decoupling logical position from UI notation.
SystemState	Represents the current execution phase of the program (e.g., MAIN_MENU_STATE, GAMEPLAY_STATE).
TTEEntry (Transposition Table Entry)	Caches previously searched position evaluation results to avoid redundant calculations in AI decision-making.
Zobrist Hashing	A hashing technique used for board-state identification to achieve $O(1)$ position lookup.

Software Architecture Overview

1.1. Major Software Components

1.1.1. Modular Hierarchy Overview



1.1.2. Root-Level Project Structure

The AnteaterChess project follows a layered and modular architecture at the root level, separating implementation, interface definitions, resources, build outputs, testing artifacts, and documentation.

The root-level structure is designed to:

- isolate implementation from interface definitions
- separate source code, generated binaries, and documentation
- support independent module development and testing
- enable a clean and maintainable build pipeline

The root-level components are described as follows:

- **src/ — Core Implementation Layer**
 - The src/ directory contains all implementation files (.c) of the system.
 - It is the core of the application and is further divided into multiple submodules, including UI, game logic, AI, input handling, system services, and data representation.
 - Each subdirectory inside src/ corresponds to a functional subsystem of the application.
- **include/ — Interface Layer**
 - The include/ directory contains all header files (.h) that define the public interfaces of each module.
 - These headers expose function declarations, data structures, and shared types, allowing different modules to interact while keeping their implementations encapsulated.
- **assets/ — UI Resource Layer**
 - The assets/ directory stores optional static resources used by the application, such as UI-related assets, images, or other supporting files.
 - This directory supports presentation and interaction features but is not part of the core logic of the system.
- **bin/ — Build Output and Runtime Artifact Layer**
 - The bin/ directory stores generated executable files and runtime-related artifacts.
 - After building the project, the main executable chess is placed in this directory.

- bin/tests/ contains compiled test executables generated during the build process.
 - bin/logs/ stores log files produced during program execution and testing.
- **tests/ — Test Source Layer**
 - The tests/ directory contains unit test source files corresponding to different modules.
 - These files are used to verify system correctness.
 - The compiled versions of these tests are placed in bin/tests/.
- **Makefile — Build Configuration**
 - The Makefile defines the build process of the project.
 - It specifies how source files are compiled and linked, enabling automated compilation through commands such as "make".
- **README — Project Overview**
 - The README file provides general information about the project, including its purpose, build instructions, usage guidelines, and a high-level overview of the system.
 - It serves as the primary entry point for developers and users.
- **INSTALL — Installation Guide**
 - The INSTALL file provides instructions for setting up, building, and running the project in a target environment.
- **COPYRIGHT — Copyright Information**
 - The COPYRIGHT file contains ownership and licensing information for the project.
- **doc/ — Project Documentation**
 - The doc/ directory stores project documentation related to system design and usage.
 - Chess_SoftwareSpec.pdf provides a detailed description of the software architecture, including

- module decomposition, data structures, interface definitions, and design decisions.
 - Chess_UserManual.pdf provides instructions for using the system from a user perspective.
 - These documents serve as references for development, maintenance, and user guidance.
-

1.1.3. Module Organization

The Anteater Chess system adopts a functionality-driven modular architecture, where each source directory corresponds to a distinct system responsibility.

Modules are organized by role rather than by file type. Each directory contains implementation files (.c) that collectively implement one cohesive subsystem.

This design ensures:

- High cohesion within modules
- Low coupling across modules
- Clear separation between data, control, gameplay, and interface layers
- Direct traceability from user-visible behavior to implementation modules

Module Hierarchy

The source code is organized as follows:

- **Program Entry (main_cli.c)**

This is the current executable entry point used by the build system.

Responsibilities:

- Initialize CLI runtime behavior
- Enter the standalone CLI application loop
- Launch the user-facing program built as bin/chess

Note:

- main.c remains in the repository as an older controller-oriented entry path, but it is not the current build entry used by the top-level Makefile.

- **Core Data Layer (core/)**

This layer defines the fundamental data structures shared across the system.

It provides representation and state containers, but does not own the full gameplay pipeline.

Files:

- position.c
- piece.c
- board.c
- move.c
- movelist.c
- player.c
- gamestate.c
- gameconfig.c

Responsibilities:

- Represent board state, pieces, players, and positions
- Define move and move-history structures
- Maintain the global game-state container and configuration objects

- **Gameplay Logic Layer (gameplay/)**

This layer implements the core gameplay pipeline, including validation, move generation, execution, and endgame analysis.

Files:

- rules.c — validates moves and selection requests
- movegen.c — generates legal move candidates
- execution.c — applies and undoes moves
- endgame.c — detects check, checkmate, draw, and termination conditions

Responsibilities:

- Enforce gameplay rules consistently
- Support both human and AI move generation
- Maintain correctness of game progression

- **Turn Management (turn/)**

This module manages player turns and per-turn timing.

Files:

- turn.c — controls turn switching
- turn_timer.c — manages countdown timers for each turn

Responsibilities:

- Track the active side to move
- Manage turn switching
- Maintain per-turn countdown behavior

- **Time System (time/)**

This module maintains global time tracking.

Files:

- clock.c

Responsibilities:

- Track total elapsed gameplay time
- Support pause/resume behavior
- Provide timing data for UI display, logging, and summaries

- **Event System (event/)**

This module provides the event-driven communication primitives used by the control flow.

Files:

- event.c
- event_queue.c

Responsibilities:

- Represent user, system, and gameplay actions as events
- Maintain prioritized event queues for processing

- **Control Layer (control/)**

This layer coordinates system behavior through the controller loop and FSM logic.

Files:

- controller.c

- fsm.c

Responsibilities:

- Process events through deterministic state transitions
- Coordinate system state changes
- Support the shared execution model used by the CLI runtime

- **Input Processing (input/)**

This module handles user input and converts it into structured commands.

Files:

- input.c — captures UI or mouse input
- command.c — parses and constructs command objects

Responsibilities:

- Normalize user input into a unified format
- Bridge UI interaction and system events

- **AI Module (ai/)**

This module generates moves for computer-controlled players.

Files:

- ai.c

Responsibilities:

- Select moves using iterative-deepening alpha-beta search
- Evaluate positions using material and positional heuristics
- Maintain the transposition-table-based search support
- Provide hint functionality through the same engine

- **Logging System (log/)**

This module manages persistent recording of gameplay.

Files:

- log.c

Responsibilities:

- Record validated move history to persistent log files
- Rebuild logs after undo operations
- Write runtime logs under bin/logs/

- **Error System (error/)**

This module handles error code and error messages.

Files:

- error.c

Responsibilities:

- Map error codes to human-readable messages
- Support consistent error reporting across the system

- **User Interface Layer (ui/)**

This layer is responsible for rendering all user-visible components.

Files:

- main_menu.c
- game_mode_menu.c
- game_setup_menu.c
- gameplay_ui.c
- board_renderer.c
- move_input_widget.c
- error_popup.c
- fatal_error_window.c
- endgame_menu.c

Responsibilities:

- Render game state and menus
- Handle user interactions visually
- Display feedback such as errors and game results

Note:

- The ui/ layer is currently present in the repository, but it is not part of the active build path defined by the top-level Makefile.

- **CLI Interface Layer (cli/)**

This is the current active user-facing interface layer used by the build.

Files:

- cli_app.c
- cli_menu.c
- cli_gameplay.c
- cli_renderer.c
- cli_feedback.c

Responsibilities:

- Render the text-based game interface
- Display menus, gameplay state, board output, and feedback
- Collect user-facing CLI interactions
- Orchestrate the standalone CLI application flow

Note:

- CLI is only for Alpha Release, so there is no further documentation for this module.
-

1.2. Main Data Structures Overview

The system follows a layered dependency structure:

- Low-level data structures (**Position**, **Piece**, **Board**) define the game environment
 - **Move** builds upon these low-level structures to represent gameplay actions
 - **Input Command** acts as an external interface and is translated into an **Event**
 - **Event** encapsulates all system interactions and serves as input to the FSM
 - **Game Configuration** represents user-defined setup parameters and provides the initialization input for the system
 - **GameState** aggregates core gameplay data and control information required for game execution, and is constructed using **Game Configuration**
 - System services (**Log**, **Timer**, **Move History**) operate on **GameState**
-

[Position](#)

Represents a coordinate on the board.

- Used to identify the location of a piece

- Serves as the basis for move definition
-

Piece

Represents a chess unit with a specific type and ownership.

- The type determines movement behavior
 - The ownership determines which player controls the piece
-

Board

Represents the game environment as a structured grid of tiles on which pieces are placed and moved.

- Defines spatial boundaries of gameplay
 - Provides positional context for all pieces
-

Move

Represents a complete gameplay action performed by a piece.

- Used as the core unit of gameplay actions and event payloads
 - May include captures, special moves, and anteater multi-capture paths
 - Depends on **Position** and **Piece**
-

Event

Represents a discrete system-level action processed by the control loop.

- Serves as the unified input to the FSM
 - Enables a unified event-driven execution model
 - Encapsulates all interactions:
 - User actions (move input, undo, exit)
 - AI-generated moves
 - System-generated triggers (timer, game over)
-

System State

Represents the current execution phase of the program.

- Controls system behavior at a high level
 - Determines how incoming events are processed
 - Defines valid transitions between states
-

Player

Represents a participant in the game.

- Each player controls a set of pieces
 - Players take turns to perform moves
-

GameConfig

Represents the set of user-defined parameters selected during the game setup phase.

- Includes game mode, player side assignment, AI difficulty levels, and timer settings

- Serves as the initialization input used to construct the GameState before gameplay begins
-

Game State

Represents the core gameplay state of the system.

- Includes all data required to describe the current game configuration and progression, such as the board, players, turn information, and system state (FSM)
 - Other runtime subsystems, such as the event queue, timer system, and logging system, are managed independently and interact with GameState through the event-driven architecture
-

Move List

Represents a generic container for storing a collection of Move objects.

- Maintains chronological order of moves
-

Log System

Represents the persistent record of the game.

- Stores moves in a human-readable format
 - Used for post-game analysis and replay
 - Consumes **Move History** and **Game State** updates
-

Timer System

Represents time tracking mechanisms associated with gameplay.

- Timer System includes two distinct components:
 - Global Clock:
 - Tracks total elapsed gameplay time.
 - Supports pause/resume based on system state.
 - Turn Timer:
 - Tracks remaining time for each player's turn.
 - Resets and updates on turn transitions.
-

Command

Represents raw user input before it is translated into a system event.

- Acts as an intermediate representationWhen events are dequeued, they are evaluated according to a predefined priority order (e.g., fatal errors, control events, system events, gameplay events).
 - Used only within the input processing module
 - Translated into an **Event** before entering the control loop
-

1.3. Overall Program Control Flow

The AnteaterChess system follows a hybrid control model that combines an event-driven architecture with a loop-based execution model.

The system is structured as a state-driven loop, where user interactions and system-generated actions are treated as events. These events are continuously monitored and processed within a central control loop.

This design ensures:

- Responsiveness to user actions at all times
 - Consistent processing of moves through a unified pipeline
 - Clear separation between control flow and game logic
-

1.3.1. System States Definition

The system is modeled as a finite-state machine (FSM), where each state represents a distinct phase of execution.

Each state processes events through a control loop, with the **GAMEPLAY_STATE** containing a continuous loop, while menu states operate as event-driven wait loops.

The following states are defined:

- **INIT_STATE**

The program initializes system resources and GameState.

- **MAIN_MENU_STATE**

The system displays the main menu and waits for user selection.

- **GAME_MODE_SELECTION_STATE**

The user selects the game mode.

- **GAME_SETUP_STATE**

The user configures game parameters (side, AI level, timer).

- **GAMEPLAY_STATE**

The core gameplay loop is active, and events are continuously processed.

- **GAME_TERMINATION_STATE**

The system stops gameplay and prepares the endgame interface.

- **END_GAME_MENU_STATE**

The system displays the **End Game Menu** and waits for user decision.

- **EXIT_STATE**

The program terminates.

Global Clock Policy:

- The global clock is active only during GAMEPLAY_STATE.
- During non-gameplay states, including MAIN_MENU_STATE, GAME_MODE_SELECTION_STATE, GAME_SETUP_STATE, GAME_TERMINATION_STATE, and END_GAME_MENU_STATE, the global clock remains paused.

1.3.2. Event Definition

External system interactions are modeled as events and processed by the FSM.

Internal state updates, such as turn switching and game termination checks, may be handled directly within the FSM logic without generating additional events.

Event Categories and Types:

- **Control Events (Highest Priority)**
 - Leave Game Event
 - Back Event
 - Exit Program Event
 - New Game Event
- **Gameplay Events**
 - Move Input Event
 - AI Move Event
 - Undo Event
- **System Events**
 - Timer Expired Event

- **Error Events** (non-fatal)
- **Fatal Error Events** (treated as control events and inserted into ControlQueue)

Event Queue Mapping:

Each event type is assigned to a specific queue:

- **ControlQueue:**
 - Leave Game Event
 - Back Event
 - Exit Program Event
 - New Game Event
 - Fatal Error Event
 - Error Event
- **SystemQueue:**
 - Timer Expired Event
- **GameplayQueue:**
 - Move Input Event
 - AI Move Event
 - Undo Event

1.3.3. Control Model Overview

The system operates as a finite-state control loop with an event queue mechanism.

Events generated from multiple sources are inserted into a set of priority-based event queues rather than a single centralized queue.

The system maintains three queues:

- ControlQueue (highest priority)
- SystemQueue
- GameplayQueue (lowest priority)

The FSM consumes events according to queue priority, always processing higher-priority queues before lower-priority ones.

This design eliminates the need for dynamic priority evaluation during

event processing and ensures deterministic behavior.

Lower-priority events may be deferred or ignored depending on system conditions.

At any point during execution, the system is in one of the states defined in [Section 1.3.1](#). State transitions are triggered by events retrieved from the event queue.

Within each state, a control loop continuously:

1. Collects events from all sources (UI, AI, timer)
2. Classifies events based on type
3. Enqueues events into the corresponding priority queue
4. Processes events via the FSM in priority order

The controller is responsible for driving the main execution loop, including polling event sources, enqueueing events, and dispatching them for processing. The FSM is the only module that performs state-dependent event handling and state transitions.

The lifecycle of the global clock is also controlled at the control-flow level. When the system enters `GAMEPLAY_STATE`, the clock is resumed. When the system leaves `GAMEPLAY_STATE`, the clock is paused.

1.3.4. Event-Driven Execution Model

External interactions in the system are modeled as events.

Event sources include:

- User input (mouse clicks, command input, button actions)
- AI-generated moves
- Timer update

All generated events are inserted into their corresponding priority queues.

The system processes events by always selecting from the highest-priority non-empty queue, ensuring that critical events are handled first.

User input is captured through non-blocking polling and converted into high-level events that are inserted into the event queue.

This queue-based mechanism ensures:

- Decoupling between event generation and processing
- Deterministic execution order

- Consistent behavior across different event sources

The system uses an event-polling mechanism to collect events and enqueue them, while a centralized FSM consumes and processes them.

This approach simulates asynchronous responsiveness in a single-threaded design.

1.3.5. Top-Level Control Flow

The overall program execution follows a state-transition structure:

1. Program Initialization
2. Main Menu Loop
3. Game Setup Loop
4. Gameplay Loop
5. Game Termination Handling

After termination, control returns to the **Main Menu** unless the user exits the program.

The control loop is implemented by the controller module, while all state transitions and state-dependent event handling are delegated to the FSM.

Clock behavior across phases:

The global clock is initialized during program initialization.

It is resumed when gameplay begins and paused whenever execution leaves active gameplay, such as when entering menus, dialogs, or termination handling states.

1.3.6. Gameplay Loop

The **Gameplay State** is governed by a central loop that continuously processes events. Events are consumed in priority order across multiple queues

All gameplay-related events are processed through a unified move pipeline, ensuring consistent behavior for both human and AI players.

Execution Model:

The gameplay loop operates using a non-blocking input model.

User input is polled continuously without pausing system execution.

The system does not wait for user input before proceeding to the next iteration of the loop.

Event processing follows a priority order enforced by queue selection:

1. **ControlQueue (including fatal errors)**
2. **SystemQueue**
3. **GameplayQueue**

This priority model ensures that high-priority control actions are handled immediately.

The loop terminates when:

- A game-ending condition is detected, or
- A user-initiated exit is confirmed

Core loop structure:

while (systemState == GAMEPLAY_STATE):

0. if a Fatal Error is detected:
 - Generate EVENT_FATAL_ERROR
 - Pass to controller
 - Controller: enqueue into ControlQueue
1. Collect external events and update internal conditions:
 - updateClock() updates the global elapsed-time clock
 - UI input → generate Move Input Event / Undo Event → pass to controller
 - AI module → generate AI Move Event → pass to controller
 - Turn timer → generate Timer Expired Event → pass to controller

Controller:

- classify events based on type
- enqueue into ControlQueue, SystemQueue, or GameplayQueue

2. Process events from the event queue:
while (any event queue is not empty):

if ControlQueue is not empty:
Event e = dequeueControlEvent(queue)

else if SystemQueue is not empty:
Event e = dequeueSystemEvent(queue)

else:
Event e = dequeueGameplayEvent(queue)

- i. High-priority control events:
 - Leave Game Event
 if Leave Game Event:
 - Transition to
GAME_TERMINATION_STATE
 - ii. System events:
 - Timer Expired Event:
 - Update timer state
 - Mark current turn as expired
 - Delegate turn transition to game logic
 - iii. Gameplay events:
 - Move Input Event / AI Move Event:
 - Construct Move object
 - Pass to move pipeline
 - If invalid:
 - Trigger an error event
 - Continue
 - Apply move and update GameState
 - Record move into MoveList
 - Update log file if needed
 - Undo Event:
 - Restore previous state from MoveList
 - Rebuild log file
3. Render current GameState (UI layer)
 4. Perform post-processing:
 - Update MoveList (memory layer)
 - Update log file (file layer)
 - Update turn indicator and timer

1.3.7. Game Termination Flow

The system enters the **GAME_TERMINATION_STATE** under two conditions:

- A termination-related control event is processed by the FSM (e.g., Leave Game Event), or
- A game termination condition is detected by the gameplay logic (e.g., checkmate, draw)

External control events are enqueued and processed by the FSM, while internally detected termination conditions trigger a direct state transition without generating additional events.

The **GAME_TERMINATION_STATE** serves as a unified termination point for all game-ending scenarios.

Behavior in a GAME_TERMINATION_STATE:

All behaviors in this state are controlled by the FSM, and no further gameplay events are processed.

- All gameplay interactions are disabled
- The move input and execution are stopped
- AI move generation is halted
- Turn management is halted
- The final Game State is preserved
- The global clock is paused in GAME_TERMINATION_STATE and remains paused throughout END_GAME_MENU_STATE.

After the GAME_TERMINATION_STATE stabilizes:

- If a termination condition has been confirmed or a Leave Game Event has been processed:
 - The FSM triggers a transition to END_GAME_MENU_STATE

END_GAME_MENU_STATE Behavior:

The End Game Menu is displayed as a modal interface layered above the Gameplay UI.

User selections generate corresponding events, which are enqueued and processed by the FSM.

The user is presented with the following options:

- **New Game**
 - Generates a New Game Event
 - Transition to GAME_MODE_SELECTION_STATE
- **Back to Main Menu**
 - Generates a Back Event
 - Transition to MAIN_MENU_STATE

Exit Condition:

If the user selects **Quit Game** at the **Main Menu**:

- An Exit Program Event is generated and enqueued
- The FSM processes the event and transitions to EXIT_STATE
- Program execution terminates

2. Installation

2.1. System Requirements and Compatibility

The Anteater Chess system is designed to run on a Linux-based environment and depends on a standard C development toolchain and graphical libraries.

Minimum requirements:	
Operating System	Linux (RHEL8-x86_64)
Architecture	x86_64
Build System	GNU Make 4.2.1 or later
Compiler	GCC 8.5.0 or later
Libraries	• SDL2 • SDL2_image • SDL2_ttf
Runtime Environment	X11-compatible display system

2.2. Setup and Configuration

2.2.1. Installation from Binary Package (User Mode)

Step 1: Obtain the binary package

Download Chess_Alpha.tar.gz.

Step 2: Extract the archive

```
$ gtar xvzf Chess_Alpha.tar.gz
```

Step 3: Enter the package directory

```
$ cd Chess_Alpha
```

Step 4: Run the program

```
$ ./bin/chess
```

2.2.2. Build from Source (Developer Mode)

Step 1: Extract the source package

```
$ gtar xvzf Chess_Alpha_src.tar.gz
```

Step 2: Navigate to project root

```
$ cd Chess_Alpha_src
```

Step 3: Build the system

```
$ make
```

Step 4: Make and run unit tests

```
$ make test
```

Step 5: Run the program

```
$ ./bin/chess
```

2.2.3. Configuration Notes

- The top-level Makefile defines the build process for the current CLI release.
 - It compiles the maintained source modules and links them into the executable stored in bin/.
 - Header files in include/ provide the public interfaces used for modular compilation.
 - Runtime logs are generated in bin/logs/.
 - Test binaries and temporary test outputs may be generated in bin/tests/.
 - The assets/ directory is reserved for optional UI resources and is not part of the current core CLI execution path.
-

2.3. Building, Compilation, and Installation

The system uses a Makefile-driven build process to compile and link the maintained modules.

Compilation Process:

- Source files (.c) are compiled into object files in the build directory.
- Object files are linked into a single executable.
- The final executable is generated as bin/chess.

Module Integration:

- Core data, gameplay logic, input handling, AI, control, logging, and CLI modules are compiled as separate subsystems.
- Interfaces defined in include/ enable inter-module communication while preserving modular structure.

Testing:

- Unit tests in tests/ are compiled and executed through:
\$ make test

Recommended verification command:

\$ make clean all test

Packaging:

- The source package archive can be regenerated through:
\$ make tar
-

2.4. Uninstallation

To uninstall the binary package:

\$ rm -rf Chess_Alpha/

To uninstall the source package:

\$ rm -rf Chess_Alpha_src/

3. Documentation of Packages, Modules, and Interfaces

3.1. Main Data Types and Structures

This section defines the concrete data structures used to implement the conceptual entities described in [Section 1.1](#). These structures form the foundation of system behavior and are shared across all modules.

3.1.1. [Board Representation](#)

```
#define ROWS 8
#define COLS 10

typedef struct {
    Piece cells[ROWS][COLS];
} Board;
```

Description:

- Fixed-size 2D array representing the chessboard

- Each cell stores a Piece value
- Empty tiles are represented using EMPTY_PIECE

Design Rationale:

- Enables constant-time access ($O(1)$)
- Avoids dynamic memory overhead
- Simplifies iteration for move generation and validation

Usage:

- Used by move validation and rule engine
- Accessed by UI for rendering
- Used by AI for board evaluation

3.1.2. [Piece Structure](#)

```
typedef enum {
    ANT, ROOK, KNIGHT, BISHOP, QUEEN, KING,
    ANTEATER, EMPTY_PIECE
} PieceType;

typedef enum {
    WHITE, BLACK, EMPTY_COLOR
} Color;

typedef struct {
    PieceType type;
    Color color;
} Piece;
```

Description:

- Represents a single chess unit
- Combines type and ownership

Design Rationale:

- Using enums ensures type safety and readability
- EMPTY_PIECE avoids the need for null pointers
- Compact representation supports efficient copying

Usage:

- Stored in Board cells
- Used in move validation and rule logic
- Used by AI for evaluation

3.1.3. [Position](#)

```
typedef struct {  
    int row;  
    int col;  
} Position;
```

Description:

- Represents a coordinate on the board

Design Rationale:

- Simple integer representation ensures fast computation
- Decouples logical position from UI notation

Usage:

- Used in Move structure
- Used in command parsing
- Used in highlighting and validation

3.1.4. [Move Representation](#)

```
typedef enum {  
    NO_SPECIAL_MOVE,  
    CASTLING_KINGSIDE,  
    CASTLING_QUEENSIDE,  
    EN_PASSANT,  
    PROMOTION_QUEEN,  
    PROMOTION_ROOK,  
    PROMOTION_BISHOP,  
    PROMOTION_KNIGHT,  
    ANTEATER_CAPTURE  
} SpecialMove;  
  
typedef struct {  
    Position pos;  
    Piece piece;  
} CaptureRecord;  
  
#define MAX_CHAIN 10
```

```
typedef struct {
    Position from;
    Position to;
    Piece movedPiece;
    Position path[MAX_CHAIN];
    int pathLength;
    CaptureRecord captures[MAX_CHAIN];
    int captureCount;
    SpecialMove specialType;
} Move;
```

Description:

- Represents a complete gameplay action performed by a single piece

Design Rationale:

- Provides a unified representation for both standard and special moves
- Stores sufficient information to support move execution, logging, replay, and undo
- Explicitly records path and capture data for anteater multi-capture moves
- CaptureRecord is used to store both the position and piece information of captured pieces, enabling accurate undo and replay of multi-capture moves

Usage:

- Used in move validation and move execution
- Stored in MoveList as the in-memory move history
- Used by the logging and undo systems

3.1.5. [Command](#)

```
typedef struct {
    Position from;
    Position to;
} Command; typedef enum {
    CMD_MOVE,
    CMD_UNDO,
    CMD_EXIT,
```

```

        CMD_BACK,
        CMD_NEW_GAME,
        CMD_INVALID
    } CommandType;

    typedef struct {
        CommandType type;
        Position from;
        Position to;
    } Command;

```

Description:

- Represents parsed user input

Design Rationale:

- Decouples UI input format from internal move representation
- Allows flexible input methods (keyboard, mouse)
- Prevents direct dependency between UI and game logic

Usage:

- Command is translated into Event before entering the control loop

3.1.6. [Event](#)

```

    typedef enum {
        EVENT_MOVE_INPUT,
        EVENT_AI_MOVE,
        EVENT_UNDO,
        EVENT_TIMER_EXPIRED,
        EVENT_LEAVE_GAME,
        EVENT_BACK,
        EVENT_NEW_GAME,
        EVENT_EXIT_PROGRAM,
        EVENT_ERROR,
        EVENT_FATAL_ERROR,
        EVENT_HINT,
        EVENT_NONE
    } EventType;

    typedef struct {

```

```

        EventType type;
        union {
            Move move;
            Command command;
            ErrorCode errorCode;
        } data;
    } Event;

```

Description:

- Represents a high-level system action that is enqueued into the EventQueue and processed by the FSM.

Design Rationale:

- Unifies all system interactions under a single data type
- Supports event-driven control flow
- Supports optional payload data depending on event type

Usage:

- The EventType determines which union field is valid:
 - EVENT_MOVE_INPUT → command
 - EVENT_AI_MOVE → move
 - EVENT_ERROR / EVENT_FATAL_ERROR → errorCode

3.1.7. [Event Queue](#)

```

typedef enum {
    QUEUE_CONTROL,
    QUEUE_SYSTEM,
    QUEUE_GAMEPLAY
} QueueType;

#define MAX_EVENTS 128

typedef struct {
    Event events[MAX_EVENTS];
    int front;
    int rear;
} SingleEventQueue;

typedef struct {
    SingleEventQueue controlQueue;

```

```
SingleEventQueue systemQueue;  
SingleEventQueue gameplayQueue;  
} EventQueue;
```

Description:

- Stores pending events in categorized queues before they are processed by the control loop.
- Events are separated into multiple queues based on their semantic roles, allowing structured handling of different event types.

Design Rationale:

- Separates event streams into distinct categories to improve clarity and control over event processing:
 - controlQueue: High-level control events (e.g., menu navigation, exit requests)
 - systemQueue: System-generated events (e.g., timer expiration, fatal errors)
 - gameplayQueue: Gameplay-related events (e.g., move input, AI actions)
- This design:
 - Enables prioritized handling of events without requiring complex priority sorting
 - Improves maintainability by grouping events by responsibility
 - Supports deterministic execution by enforcing a fixed processing order across queues

Usage:

- All events are passed to the controller, which determines the appropriate target queue based on event type and enqueues them into ControlQueue, SystemQueue, or GameplayQueue.
- The control loop processes events in a defined priority order, for example:
 - controlQueue → systemQueue → gameplayQueue
- Within each queue, events are processed in FIFO order
- The system ensures that higher-priority events are handled before lower-priority ones without mixing event types

3.1.8. [Player](#)

```
typedef enum {  
    HUMAN, AI
```

```

    } PlayerType;

    typedef struct {
        PlayerType type;
        Color color;
    } Player;

```

Description:

- Represents a participant in the game

Design Rationale:

- Supports flexible game modes
- Separates control logic from game rules

Usage:

- Stored in GameState
- Used in turn management
- Used in AI decision flow

3.1.9. [System State](#)

```

typedef enum {
    INIT_STATE,
    MAIN_MENU_STATE,
    GAME_MODE_SELECTION_STATE,
    GAME_SETUP_STATE,
    GAMEPLAY_STATE,
    END_GAME_MENU_STATE,
    GAME_TERMINATION_STATE,
    EXIT_STATE
} SystemState;

```

Description:

- Represents the current execution state of the FSM.

Design Rationale:

- Provides explicit control over system flow

Usage:

- Stored in GameState
- Used by the FSM to control system flow
- Determines active UI and gameplay behavior

3.1.10. [Game Configuration](#)

```
typedef enum {  
    MODE_HUMAN_VS_HUMAN,  
    MODE_HUMAN_VS_COMPUTER,  
    MODE_COMPUTER_VS_COMPUTER  
} GameMode;  
  
typedef enum {  
    DIFFICULTY_NONE,  
    DIFFICULTY_EASY,  
    DIFFICULTY_MEDIUM,  
    DIFFICULTY_HARD  
} AIDifficulty;  
  
typedef struct {  
    GameMode mode;  
    Color playerColor;  
    AIDifficulty aiDifficultyWhite;  
    AIDifficulty aiDifficultyBlack;  
    int timerEnabled;  
    int aiTimeLimit;  
    int initialTimeSeconds;  
} GameConfig;
```

Description:

- GameConfig encapsulates all user-defined configuration parameters selected during the game setup phase.

Design Rationale:

- Separating GameConfig from GameState establishes a clear distinction between configuration input and runtime state
- This design improves modularity by isolating setup-related parameters from dynamic gameplay data, reducing coupling between the UI layer and core logic
- Using a unified configuration structure ensures consistent initialization across different game modes, simplifies parameter passing during setup, and supports extensibility for future features such as additional game rules or advanced timing options

- Including DIFFICULTY_NONE allows the system to explicitly represent non-AI participants, avoiding ambiguous or unused values in Human vs Human mode

Usage:

- GameConfig is constructed in the UI layer during the GAME_SETUP_STATE through user interaction
- It is passed to initGameState() to initialize the GameState, including player roles, AI configuration, and timer system setup
- The controller uses GameConfig to determine:
 - whether a player is controlled by a human or AI
 - which AI difficulty level to apply
 - whether the timer system should be activated
- After initialization, GameConfig is not modified and is not used as part of the runtime game state

3.1.11. [Move List](#)

```
#define MAX_MOVES 1024

typedef struct {
    Move moves[MAX_MOVES];
    int count;
} MoveList;
```

Description:

- Represents a generic container for storing a collection of Move objects.
- It is used across multiple stages of the move pipeline, including move generation, move validation, AI decision-making, and game history storage
- Depending on the context, MoveList may represent either a transient set of candidate moves or a persistent record of executed moves

Design Rationale:

- MoveList is designed as a reusable container to avoid duplicating data structures for different stages of move processing
- This unified design reduces implementation complexity and ensures consistent handling of Move objects across modules such as move generation, rules validation, AI evaluation, and logging.
- Although MoveList is reused in both transient and persistent contexts, its role is clearly defined by the module in which it is

used, preserving semantic clarity without introducing additional structural overhead

Usage:

- MoveList is used in different contexts throughout the system:
 - In move generation modules, it stores pseudo-legal or candidate moves
 - In rules modules, it represents filtered legal moves.
 - In the AI module, it serves as the input set for move evaluation and selection
 - In GameState, it is used as moveHistory to record all executed moves during gameplay
 - The interpretation of MoveList depends on the calling context, and its lifecycle is managed by the corresponding module
-

3.1.12. [Game State](#)

```
#include <stdint.h>

typedef enum {
    RESULT_NONE,
    RESULT_WHITE_WIN,
    RESULT_BLACK_WIN,
    RESULT_DRAW,
    RESULT_TERMINATED_BY_USER
} GameResult;

typedef struct {
    Board board;
    Player players[2];
    Color currentTurn;
    int moveCount;
    int gameOver;
    GameResult result;
    SystemState systemState;
    GameConfig config;
    MoveList moveHistory;
    uint64_t hash;
} GameState;
```

Description:

- Represents the complete runtime state of an active game session, including board configuration, player information, progression status, and final game outcome.

Design Rationale:

- Centralizes the core gameplay state in a single structure
- Reduces state fragmentation across major gameplay modules
- Ensures consistent access to board, turn, and control-state information
- Supports FSM-driven execution and predictable state transitions
- Keeps gameplay state logically separated from system services such as timer, logging, and event management, while allowing controlled data access through well-defined interfaces
- currentTurn indicates which player (WHITE or BLACK) is currently active
- Stores the final game result explicitly to support:
 - End Game Menu rendering
 - Move Log output
 - Consistent termination handling across all game-ending scenarios

Usage:

- Shared across gameplay-related modules for reading and updating the current game state
- Used by the UI for rendering and by the logic and AI modules for decision-making
- Modified through FSM-driven event processing

3.1.13. [Log System \(File Layer\)](#)

```
typedef struct {
    FILE *file;
} LogFile;
```

Description:

- Handles persistent storage

Design Rationale:

- Separates memory state from file I/O
- Ensures human-readable output

Usage:

- Writes moves after validation
- Acts as a persistent record of gameplay

- Not used as the primary source for undo
-

3.1.14. [Timer System](#)

```
typedef struct {
    int hours;
    int minutes;
    int seconds;
} Time;

typedef struct {
    Time elapsedTime;
} GlobalClock;

typedef struct {
    Time remainingTime;
    int timeLimit;
    int enabled;
    int expired;
} TurnTimer;
```

Description:

- Represents time tracking

Design Rationale:

- Separates timing logic from gameplay state
- Supports interrupt-driven event generation

Usage:

- Updated each turn
 - Used in UI display
-

3.1.15. **Transposition Table Entry**

```
#include <stdint.h>

typedef enum {
    TT_EXACT,
    TT_ALPHA,
    TT_BETA
} TTFlag;
```

```
#define TT_SIZE (1 << 20)

typedef struct {
    uint64_t hash;
    int depth;
    int score;
    TTFlag flag;
    Move best;
    int valid
} TTEntry;
```

Description:

- AI transposition table entries cache previously searched position evaluation results to avoid redundant calculations.

Design Rationale:

- Use Zobrist hashing as the key to achieve $O(1)$ position lookup
- TTFlag distinguishes between exact value, upper bound, and lower bound to adapt to Alpha-Beta pruning
- Store the best move for move ordering (hash move has the highest priority)

Usage:

- Clear the entire table when `initAI()` is called.
 - At the entry of Alpha-Beta search, look up the table; at the exit, write to the table.
 - When sorting moves, use the best field as the hash move.
-

3.2. Detailed Description of Functions and Parameters

This section documents the public interfaces of the system at the header-file level. Header files are grouped by subsystem so that each file corresponds to a cohesive module responsibility.

For each header file, this section specifies:

1. dependencies
2. module responsibility
3. required contents
4. data types defined in the file
5. function signatures with input/output specifications and return-value notes

Return Value Convention:

Unless otherwise specified, all functions returning an integer follow this convention:

- 0 indicates success
- Non-zero indicates failure

Specific error conditions may be further distinguished using defined error codes.

3.2.1. [Core Data Headers](#)

core/position.h

Responsibility

Defines the Position data type and basic utility functions for representing and manipulating board coordinates.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - (none)

Data Type Defined in This File

- [Position](#)

Function Signatures

```
Position createPosition(int row, int col);
```

Input:

- row: zero-based row index
- col: zero-based column index

Output:

- Returns a Position object initialized with the specified row and column.

Note:

- This function only constructs a Position object.
- It does not guarantee that the position is within board bounds.

```
int isValidPosition(Position pos);
```

Input:

- pos: position to validate

Output:

- Returns non-zero if pos is within the valid board range
- Returns 0 otherwise

Note:

- A return value of 0 means the position is invalid and should not be used for board access.

```
Position parsePosition(const char *input);
```

Input:

- input: coordinate string, such as "A2"

Output:

- Returns the parsed internal Position

Note:

- If parsing fails, this function returns an invalid Position.
- The caller must verify the result with isValidPosition().
- An invalid return means the input string does not represent a legal board coordinate.

```
int positionEqual(Position a, Position b);
```

Input:

- a: first position
- b: second position

Output:

- Returns 1 if the two positions are identical
- Returns 0 otherwise

Note:

- A return value of 0 means the two positions are different.
-

core/piece.h**Responsibility**

Defines all piece-related core data structures, including piece type, ownership color, and the combined Piece abstraction used throughout the system.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - (none)

Data Type Defined in This File

- [PieceType](#)
- [Color](#)
- [Piece](#)

Function Signatures

```
Piece createPiece(PieceType type, Color color);
```

Input:

- type: piece type
- color: piece color

Output:

- Returns a Piece object initialized with the specified type and color

Note:

- This function only constructs a piece object.
- It does not check gameplay legality.

```
int isSameColor(Piece a, Piece b);
```

Input:

- a: first piece
- b: second piece

Output:

- Returns 1 if both pieces have the same color
- Returns 0 otherwise

Note:

- A return value of 0 means the two pieces belong to opposite side.

```
char getPieceSymbol(Piece piece);
```

Input:

- piece: piece to convert to a display symbol

Output:

- Returns the character symbol corresponding to the piece

Note:

- If the piece is empty or invalid, the returned symbol should be interpreted according to the implementation policy for empty or fallback display.

```
int isValidPiece(Piece piece);
```

Input:

- piece: piece object to validate

Output:

- Returns 0 if the piece contains a valid type/color combination
- Returns non-zero otherwise

Note:

- A return value of 0 means the piece should be treated as a valid gameplay piece.
-

core/board.h**Responsibility**

Defines the board representation and the basic board manipulation interface, including initialization, piece lookup, piece placement, and piece removal.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [position.h](#)
 - [piece.h](#)

Data Type Defined in This File

- [Board](#)

Function Signatures

```
void initBoard(Board *board);
```

Input:

- board: pointer to the board to initialize

Output:

- Initializes the board into the standard Ant eater Chess starting configuration

Note:

- This function does not return a value.
- The caller must provide a valid non-NULL board pointer.

```
Piece getPiece(const Board *board, Position pos);
```

Input:

- board: pointer to the board
- pos: target position

Output:

- Returns the piece located at the specified position

Note:

- If the position is empty, an EMPTY piece (or equivalent representation) is returned.
- The caller should ensure the position is within board bounds.

```
void setPiece(Board *board, Position pos, Piece piece);
```

Input:

- board: pointer to the board
- pos: target position
- piece: piece to place

Output:

- Stores the specified piece at the given position

Note:

- This function does not return a value.
- The caller should ensure the position is valid.
- This operation overwrites any existing piece at the position.

```
void removePiece(Board *board, Position pos);
```

Input:

- board: pointer to the board

- pos: target position to clear

Output:

- Replaces the specified square with an empty-piece value

Note:

- This function does not return a value.
 - If pos is invalid, the operation should be ignored or guarded against by the implementation.
-

core/move.h**Responsibility**

Defines the internal representation of a move, including standard moves, special moves, capture records, and anteater multi-capture paths. Defines the internal representation of a move, including standard moves, special moves, capture records, and anteater multi-capture paths.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [position.h](#)
 - [piece.h](#)

Data Type Defined in This File

- [SpecialMove](#)
- [CaptureRecord](#)
- [Move](#)

Function Signatures

```
Move createMove(Position from, Position to, Piece  
piece);
```

Input:

- from: source position
- to: destination position
- piece: piece being moved

Output:

- Returns a Move object initialized as a basic move

Note:

- The returned move is only a data object.
- It is not guaranteed to be legal.

```
void addCapture(Move *move, Position pos, Piece piece);
```

Input:

- move: pointer to the move being updated
- pos: position of the captured piece
- piece: captured piece information

Output:

- Appends one capture record to the move

Note:

- This function does not return a value.
- If the capture array is full, the additional record should be ignored or guarded by implementation checks.

```
void addPathStep(Move *move, Position pos);
```

Input:

- move: pointer to the move being updated
- pos: next path position to append

Output:

- Appends one path step to the move

Note:

- This function does not return a value.
- If the path array is full, the additional step should be ignored or guarded by implementation checks.

```
void setSpecialMove(Move *move, SpecialMove type);
```

Input:

- move: pointer to the move
- type: special move type to assign

Output:

- Updates the specialType field of the move

Note:

- This function does not return a value.
 - It annotates the move object only and does not validate gameplay legality.
-

core/movelist.h**Responsibility**

Defines the generic container used to store collections of moves. The same structure is reused in legal move generation, AI move selection, and runtime move history.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [move.h](#)

Data Type Defined in This File

- [MoveList](#)

Function Signatures

```
void initMoveList(MoveList *list);
```

Input:

- list: pointer to the move list

Output:

- Resets the move list to an empty state

Note:

- This function does not return a value.
- The caller must provide a valid non-NULL pointer.

```
int addMove(MoveList *list, Move move);
```

Input:

- list: pointer to the move list
- move: move to append

Output:

- Returns 0 if successful
- Returns 1 if the list is at capacity

Note:

- Used by move generation, move history, and undo system.
- Failure must be handled by the caller.

```
int removeLastMove(MoveList *list);
```

Input:

- list: pointer to the move list

Output:

- Returns 0 if successful
- Returns 1 if the list is empty

Note:

- Primarily used by undo system.

```
Move *getMove(MoveList *list, int index);
```

Input:

- list: pointer to the move list
- index: zero-based move index

Output:

- Returns a pointer to the move at the specified index
- Returns NULL if index is out of bounds

Note:

- Returns NULL if list is NULL or if index is out of range.
- A NULL return means no valid move exists at the requested index.

```
int getMoveCount(MoveList *list);
```

Input:

- list: pointer to the move list

Output:

- Returns the number of moves currently stored

Note:

- Returns 0 if the list is empty.
- A return value of 0 means the list currently contains no moves.

core/player.h

Responsibility

Defines the player abstraction used in game configuration and runtime state. It distinguishes player control type from player color.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [piece.h](#)

Data Type Defined in This File

- [PlayerType](#)
- [Player](#)

Function Signatures

```
Player createPlayer(Color color, PlayerType  
type);
```

Input:

- color: player color
- type: player control type

Output:

- Returns a Player object initialized with the specified attributes

Note:

- This function only constructs the player object.
- It does not verify consistency with game mode.

```
Color getPlayerColor(Player player);
```

Input:

- player: player object

Output:

- Returns the color assigned to the player

Note:

- The returned value should be interpreted according to the Color enumeration.

```
int isAIPlayer(Player player);
```

Input:

- player: player object to test

Output:

- Returns 1 if the player is AI-controlled
- Returns 0 otherwise

Note:

- A return value of 0 means the player is not AI-controlled.

```
int isHumanPlayer(Player player);
```

Input:

- player: player object to test

Output:

- Returns 1 if the player is human-controlled
- Returns 0 otherwise

Note:

- A return value of 0 means the player is not human-controlled.
-

core/gameconfig.h

Responsibility

Defines the setup-time configuration model of the system. It stores user-selected parameters from the game setup phase and serves as the initialization input for a new game session.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [piece.h](#)

Data Type Defined in This File

- [GameMode](#)
- [AIDifficulty](#)

- [GameConfig](#)

Function Signatures

```
void initDefaultGameConfig(GameConfig *config);
```

Input:

- config: pointer to GameConfig to initialize

Output:

- Initializes the configuration with default values

Note:

- This function sets a consistent baseline configuration used for:
 - New Game initialization
 - Game Setup Menu default state
 - The caller must provide a valid non-NULL pointer.
 - This function does not perform validation.
-

core/gamestate.h

Responsibility

Defines the complete runtime state of an active game session. It is the central shared data container used by gameplay logic, turn management, control flow, AI, logging, and rendering.

Includes

- Standard Libraries:
 - <stdint.h>
- Project Headers:
 - [board.h](#)
 - [player.h](#)
 - [movelist.h](#)
 - [gameconfig.h](#)
 - [system_state.h](#)

Data Type Defined in This File

- [GameResult](#)
- [GameState](#)

Function Signatures

```
void initGameState(GameState *state, const
GameConfig *config);
```

Input:

- state: pointer to the game state to initialize
- config: pointer to the selected game configuration

Output:

- Initializes the runtime game state for a new session

Note:

- This function does not return a value.
- The caller must provide valid non-NULL pointers.

```
int isGameOver(const GameState *state);
```

Input:

- state: pointer to the game state

Output:

- Returns 1 if the game has ended
- Returns 0 if gameplay is still active

Note:

- A return value of 0 means the game isn't in a terminal state.

```
void setGameOver(GameState *state);
```

Input:

- state: pointer to the game state
- status: game-over flag to assign

Output:

- Updates the internal game-over flag

Note:

- This function does not return a value.

```
Player *getCurrentPlayer(GameState *state);
```

Input:

- state: pointer to the game state

Output:

- Returns a pointer to the player whose turn is currently active

Note:

- Returns NULL if state is NULL.
- A NULL return means no current player can be safely retrieved.

```
MoveList *getMoveHistory(GameState *state);
```

Input:

- state: pointer to the game state

Output:

- Returns a pointer to the in-memory move history

Note:

- Returns NULL if state is NULL.
- A NULL return means the move history is not accessible.

```
int addMoveToHistory(GameState *state, Move  
move);
```

Input:

- state: pointer to the game state
- move: executed move to append

Output:

- Appends the specified move to the move-history container in the game state.
- Returns 0/1 indicating whether the operation succeeds.

Note:

- Returns 0 if the move is added successfully.
- Returns 1 if the history container has reached capacity.
- This function should only be called after a move has been validated and executed.

```
void setGameResult(GameState *state, GameResult  
result);
```

Input:

- state: pointer to the game state
- result: final or current game result to record

Output:

- Updates the game result field of the runtime state

Note:

- This function does not return a value.

```
GameResult getResult(GameState *state);
```

Input:

- state: pointer to the game state

Output:

- Returns the currently stored game result

Note:

- If no result has been assigned yet, the function returns RESULT_NONE.
- RESULT_NONE means no terminal outcome has been recorded.

```
int removeLastMoveFromHistory(GameState *state);
```

Input:

- state: pointer to the game state

Output:

- Attempts to remove the most recent move from the move-history container.
- Returns 1/0 indicating whether the operation succeeds.

Note:

- Returns 0 if the last move is removed successfully.
- Returns 1 if the history container is empty.
- This function only updates the move-history container and does not modify
- the board, current turn, or other gameplay state.
- The caller must provide a valid non-NULL GameState pointer.

core/hash.h**Responsibility**

Provides hashing support for board-state identification, mainly for AI search optimization and transposition-table use.

Includes

- Standard Libraries:
 - `<stdint.h>`
- Project Headers:
 - [gamestate.h](#)
 - [move.h](#)

Data Type Defined in This File

- No new public structure is defined in this header.
- It exports hash-related utility functions.

Function Signatures

```
void initZobrist(void);
```

Input:

- No input parameters

Output:

- Initializes the internal random keys required for Zobrist hashing

Note:

- This function does not return a value.
- It should be called before any hash-based lookup is used.

```
uint64_t computeHash(const GameState *state);
```

Input:

- state: pointer to the current game state

Output:

- Returns the 64-bit hash value corresponding to the current position

Note:

- If state is invalid, the returned hash should not be treated as a reliable board identifier.

```
uint64_t updateHashMove(uint64_t hash, Move  
move);
```

Input:

- hash: previous hash value

- move: move applied incrementally

Output:

- Returns the updated 64-bit hash value after the move transformation

Note:

- The returned value is meaningful only if the input hash corresponds to the correct pre-move position and the move object is valid.

3.2.2. [Gameplay Logic Headers](#)

gameplay/execution.h**Responsibility**

Encapsulates rule enforcement and state mutation logic, including move application, capture handling, and special move resolution.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)
 - [move.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int applyMove(GameState *state, Move move);
```

Input:

- state: Pointer to the current game state
- move: Move to be applied

Output:

- Returns 0 if the move is successfully applied
- Returns non-zero if the move application fails

Note:

- Failure indicates invalid or inconsistent move
- GameState remains unchanged if the function fails

```
int undoMove(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if undo operation succeeds
- Returns non-zero if no move is available to undo

Note:

- Failure indicates empty move history or invalid undo state
-

gameplay/validation.h

Responsibility

Validates move legality and selection correctness without modifying the game state.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)
 - [move.h](#)
 - [position.h](#)

Data Type Defined in This File

- SelectionResult

```
typedef enum {
    SELECT_VALID,
    SELECT_EMPTY,
    SELECT_OPPONENT_PIECE,
    SELECT_OUT_OF_BOUNDS
} SelectionResult;
```

Function Signatures

```
int validateMove(const GameState *state, Move
move);
```

Input:

- state: Pointer to the current game state
- move: Move to validate

Output:

- Returns 0 if the move is valid
- Returns non-zero if the move is invalid

Note:

- This function performs rule checking only
- It does not modify the game state

```
SelectionResult validateSelection(const GameState
*state, Position pos);
```

Input:

- state: Pointer to the current game state
- pos: Selected board position

Output:

- Returns a SelectionResult indicating validation outcome

Note:

- This function distinguishes different selection failure reasons
 - Used for UI feedback and error messaging
-

gameplay/movegen.h**Responsibility**

Generates candidate moves for AI decision-making and hint system.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)
 - [movelist.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int generateMoves(const GameState *state,  
MoveList *list);
```

Input:

- state: Pointer to the current game state
- list: Pointer to a MoveList structure to store generated moves

Output:

- Returns 0 if move generation succeeds
- Returns non-zero if generation fails

Note:

- The MoveList must be pre-initialized before calling this function
- Failure indicates invalid state or memory issue

```
int generateLegalMoves(const GameState *state,  
MoveList *list);
```

Input:

- state: Pointer to the current game state
- list: Output container for legal moves

Output:

- Returns 0 if legal move generation succeeds
- Returns non-zero if generation fails

Note:

- Filters out illegal moves (e.g., moves leaving king in check)
- Typically used by AI and hint system

```
int generateLegalMovesForPosition(const GameState  
*state, Position from, MoveList *list);
```

Input:

- state: pointer to the game state
- from: position of the selected piece
- list: pointer to MoveList to store generated moves

Output:

- Populates the MoveList with all legal moves originating from the specified position.
- Returns 0/1 indicating success or failure.

Note:

- This function is primarily used by the UI layer to support move highlighting.
 - If the selected position does not contain a valid piece belonging to the current player, the resulting MoveList will be empty.
 - The caller must ensure that the MoveList is initialized before use.
 - The function does not clear the MoveList automatically unless specified by implementation.
-

gameplay/endgame.h

Responsibility

Generates candidate moves for AI decision-making, hint system, and UI move highlighting.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int isInCheck(const GameState *state, Color
color);
```

Input:

- Returns 1 if the specified player is in check
- Returns 0 otherwise

Output:

- Returns 0 if move generation succeeds
- Returns non-zero if generation fails

Note:

- Determines whether the king of the specified color is under attack

- This function does not modify the game state

```
int isCheckmate(const GameState *state, Color
color);
```

Input:

- state: Pointer to the current game state
- color: Player color to evaluate

Output:

- Returns 1 if the specified player is in checkmate
- Returns 0 otherwise

Note:

- A player is in checkmate if the king is in check and no legal moves are available
- Relies on move generation and validation logic
- This function does not modify the game state

```
int isStalemate(const GameState *state, Color
color);
```

Input:

- state: Pointer to the current game state
- color: Player color to evaluate

Output:

- Returns 1 if the specified player is in stalemate
- Returns 0 otherwise

Note:

- A stalemate occurs when the player is not in check and has no legal moves
- Relies on legal move generation
- This function does not modify the game state

```
int isInsufficientMaterial(const GameState
*state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 1 if the game is a draw due to insufficient material
- Returns 0 otherwise

Note:

- Checks whether neither player has sufficient material to force a checkmate
- Typical cases include king vs king, king and bishop vs king, and king and knight vs king
- This function does not modify the game state

```
int detectGameResult(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 1 if a terminal game condition is detected
- Returns 0 otherwise

Note:

- Evaluates the current game state and determines whether the game has ended
- If a terminal condition is detected:
 - The GameResult field in GameState is updated
 - The gameOver flag is set
- This function serves as the central entry point for endgame detection
- Typically invoked after each move execution

3.2.3. [Turn and Time Headers](#)

turn/turn.h

Responsibility

Manages turn progression and player switching logic. Ensures correct player ownership and synchronization of turn state across the system.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int switchTurn(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if the turn is successfully switched
- Returns non-zero if the operation fails

Note:

- Failure indicates invalid game state or game already terminated

```
Color getCurrentTurn(const GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns the current player's Color (WHITE or BLACK)
- Returns an invalid value (implementation-defined) on error

Note:

- This function does not modify the game state
- The returned value represents the owner of the current turn

```
int isPlayerTurn(const GameState *state, Color  
playerColor);
```

Input:

- state: Pointer to the current game state
- playerColor: Player color (WHITE or BLACK)

Output:

- Returns 1 if it is the specified player's turn
- Returns 0 if it is not the player's turn
- Returns negative value on error
- Returns an invalid value (implementation-defined) on error

Note:

- Used for validating move ownership before execution

```
int restoreTurnAfterUndo(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if the turn is successfully restored
- Returns non-zero if the operation fails

Note:

- Restores the turn state after an undo operation
 - This function ensures that the current turn correctly reflects the player who made the previous move
 - It is typically invoked after reverting the board state and removing the last move from history
 - The function may determine the correct turn based on:
 - the current move count, or
 - the remaining move history
 - This function modifies the game state
-

turn/turn_timer.h**Responsibility**

Handles time tracking and timeout detection for each player during gameplay. Supports countdown-based time control and integrates with the game loop.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int initTurnTimer(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if timer initialization succeeds
- Returns non-zero if initialization fails

Note:

- Initializes timer values based on game configuration
- Must be called before gameplay starts

```
int updateTurnTimer(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if timer update succeeds
- Returns non-zero if update fails

Note:

- Should be called periodically in the game loop
- Updates the active player's remaining time

```
int isTimeUp(const GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 1 if time has expired
- Returns 0 if time remains
- Returns negative value on error

Note:

- Should be called periodically in the game loop
- Updates the active player's remaining time

```
int getRemainingTime(const GameState *state,  
Color playerColor);
```

Input:

- state: Pointer to the current game state
- playerColor: Player color (WHITE or BLACK)

Output:

- Returns remaining time (in milliseconds or seconds)
- Returns negative value on error

Note:

- This function is used for UI display and time monitoring
- A negative return value indicates invalid state or invalid player color
- This function does not modify the game state

```
int resetTurnTimer(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if the turn timer is successfully reset
- Returns non-zero if the operation fails

Note:

- Resets the turn timer for the current player at the start of a new turn
 - This function is typically invoked immediately after a turn switch to initialize timing for the next player
 - It ensures that the timer state remains consistent across turns
 - This function does not affect the global clock (if present), and only operates on the turn-based timing system
 - This function modifies the game state
-

time/clock.h**Responsibility**

Maintains a global system clock used for tracking overall elapsed time independent of turn-based timing..

Includes

- Standard Libraries:
 - <stdint.h>
- Project Headers:
 - (none)

Data Type Defined in This File

- (none)

Function Signatures

```
int initClock(void);
```

Input:

- None

Output:

- Returns 0 if the clock is successfully initialized
- Returns non-zero if initialization fails

Note:

- Initializes the global clock at system start
- This function should be called once during system initialization
- The clock operates independently from the turn-based timer

```
int updateClock(void);
```

Input:

- None

Output:

- Returns 0 if the clock is successfully updated
- Returns non-zero if the update fails

Note:

- Updates the internal state of the global clock
- Typically called once per game loop iteration
- This function does not depend on the game state

```
int64_t getElapsedTimeSeconds(void);
```

Input:

- None

Output:

- Returns the total elapsed time in seconds since the clock was initialized

Note:

- Provides read-only access to the global elapsed time
- This value can be used for logging, statistics, or UI display

- This function does not modify any system state

```
int pauseClock(void);
```

Input:

- None

Output:

- Returns 0 on success, non-zero on failure

Note:

- Pauses the clock, preventing further accumulation of elapsed gameplay time
- This function modifies the internal state of the clock system
- Subsequent calls to `getElapsedTimeSeconds()` will return a constant value until `resumeClock()` is invoked
- Typically called when the system transitions into non-gameplay states (e.g., menus, dialogs)

```
int resumeClock(void);
```

Input:

- None

Output:

- Returns 0 on success, non-zero on failure

Note:

- Resumes the clock after being paused, allowing elapsed gameplay time to continue accumulating
- This function modifies the internal state of the clock system
- The elapsed time continues from the moment it was paused
- Typically called when the system returns to active gameplay

3.2.4. [Event and Control Headers](#)

system/system_state.h

Responsibility

Defines the finite-state machine (FSM) states that represent different execution phases of the system.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - (none)

Data Type Defined in This File

- [SystemState](#)
-

system/event.h

Responsibility

Defines the Event abstraction used for communication between system components and as the input to the FSM.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [move.h](#)
 - [command.h](#)
 - [error_code.h](#)

Data Type Defined in This File

- [Event](#)
- [EventType](#)

Function Signatures

```
Event createMoveInputEvent(Command cmd);
```

Input:

- cmd: Parsed user command

Output:

- Returns an Event of type EVENT_MOVE_INPUT

Note:

- The command field in the union is populated

```
Event createAIMoveEvent(Move move);
```

Input:

- move: AI-generated move

Output:

- Returns an Event of type EVENT_AI_MOVE

```
Event createSystemEvent(EventType type);
```

Input:

- type: Event type (must not require payload)

Output:

- Returns an Event with the specified type

Note:

- Used for control events such as BACK or EXIT

```
Event createUndoEvent(void);
```

Input:

- None

Output:

- Returns an Event of type EVENT_UNDO

```
Event createErrorEvent(ErrorCode code);
```

Input:

- code: Error code

Output:

- Returns an Event of type EVENT_ERROR

```
Event createFatalErrorEvent(ErrorCode code);
```

Input:

- code: Error code

Output:

- Returns an Event of type EVENT_FATAL_ERROR

system/event_queue.h

Responsibility

Buffers and organizes events into categorized queues to support prioritized and deterministic event processing.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [event.h](#)

Data Type Defined in This File

- [EventQueue](#)
- [QueueType](#)
- [SingleEventQueue](#)

Function Signatures

```
int enqueueEvent(EventQueue *q, Event e,  
QueueType type);
```

Input:

- q: Pointer to the event queue structure
- e: Event to enqueue
- type: QueueType indicating the target queue

Output:

- Returns 0 if the event is successfully enqueued
- Returns non-zero if the operation fails (e.g., queue full)

Note:

- Inserts an event into the appropriate internal queue based on its type
- This function enables explicit control over event categorization
- The caller is responsible for selecting the correct queue type

```
Event dequeueControlEvent(EventQueue *q);
```

Input:

- q: Pointer to the event queue structure

Output:

- Returns the next event from the control queue

Note:

- Removes and returns the next event in FIFO order from the control queue
- The caller should ensure the queue is not empty before calling

```
Event dequeueSystemEvent(EventQueue *q);
```

Input:

- q: Pointer to the event queue structure

Output:

- Returns the next event from the system queue

Note:

- Removes and returns the next event in FIFO order from the system queue
- Used for handling system-level events such as timer expiration or fatal errors

```
Event dequeueGameplayEvent(EventQueue *q);
```

Input:

- q: Pointer to event queue

Output:

- Returns the next event from the gameplay queue

Note:

- Removes and returns the next event in FIFO order from the gameplay queue
- Used for gameplay-related actions such as move execution

```
int isControlQueueEmpty(const EventQueue *q);
```

Input:

- q: Pointer to the event queue structure

Output:

- Returns 1 if the control queue is empty
- Returns 0 otherwise

```
int isSystemQueueEmpty(const EventQueue *q);
```

Input:

- q: Pointer to the event queue structure

Output:

- Returns 1 if the system queue is empty
- Returns 0 otherwise

```
int isGameplayQueueEmpty(const EventQueue *q);;
```

Input:

- q: Pointer to the event queue structure

Output:

- Returns 1 if the gameplay queue is empty
 - Returns 0 otherwise
-

system/fsm.h

Responsibility

Processes events retrieved from priority-based queues. It assumes that events are already ordered by priority at the queue level. Performs state-dependent logic and state transitions. The FSM is the only module responsible for modifying the SystemState.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)
 - [event.h](#)
 - [system_state.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int processEvent(GameState *state, Event event);
```

Input:

- state: Pointer to the current game state
- event: Event to process

Output:

- Returns 0 if event processing succeeds
- Returns non-zero if processing fails

Note:

- This function may update GameState and trigger state transitions

```
int transitionState(GameState *state, SystemState
newState);
```

Input:

- state: Pointer to the current game state
- newState: Target system state

Output:

- Returns 0 if transition succeeds
- Returns non-zero if transition is invalid

Note:

- Only valid transitions defined by the FSM should be allowed
-

[system/controller.h](#)

Responsibility

- Collect events from UI, AI, and timer subsystems
- Classify events based on type
- Enqueue events into ControlQueue, SystemQueue, or GameplayQueue
- Drive the main loop

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)
 - [event_queue.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int runGameLoop(GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if execution completes normally
- Returns non-zero if a fatal error occurs

Note:

- Executes the main game loop until the game reaches a terminal state or exits
 - This function is responsible for:
 - Polling events
 - Dispatching events to appropriate handlers
 - Updating the game state
 - Coordinating interactions between UI, gameplay logic, AI, and system services
 - The loop continues until a termination condition is detected
 - This function modifies the game state
-

3.2.5. [Input Headers](#)

input/command.h**Responsibility**

Defines the Command abstraction used to represent structured user input. Encapsulates user intentions such as move input, undo, exit, and menu navigation.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [position.h](#)

Data Type Defined in This File

- [Command](#)
- [CommandType](#)

Function Signatures

```
int createMoveCommand(Command *cmd, Position
from, Position to);
```

Input:

- cmd: Pointer to Command to populate
- from: Source position
- to: Destination position

Output:

- Returns 0 if command creation succeeds
- Returns non-zero if input is invalid

Note:

- Initializes a CMD_MOVE command
- If return value is non-zero, cmd content is undefined

```
int createSimpleCommand(Command *cmd, CommandType  
type)
```

Input:

- cmd: Pointer to Command
- type: Command type (must not be CMD_MOVE)

Output:

- Returns 0 if creation succeeds
- Returns non-zero if type is invalid

Note:

- Used for commands such as UNDO, EXIT, BACK
 - If return value is non-zero, cmd is not modified
-

input/input.h**Responsibility**

Provides the interface for collecting user input from different interaction modes (command line or board interaction). Acts as the entry point for all user-generated input before it is converted into events.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [command.h](#)
 - [position.h](#)

Data Type Defined in This File

- InputType

```
typedef enum {
    INPUT_SELECT_PIECE,
    INPUT_SELECT_DESTINATION,
    INPUT_CANCEL
} InputType;
```

Function Signatures

```
int getMoveInput(Command *cmd);
```

Input:

- cmd: Pointer to Command

Output:

- Returns 0 if input is successfully parsed
- Returns non-zero if input is invalid or incomplete

Note:

- Produces a CMD_MOVE command
- If return value is non-zero, cmd should not be used

```
int getBoardInput(Position *pos, InputType
*type);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if timer update succeeds
- Returns non-zero if update fails

Note:

- Used for GUI-style interaction (click-based input)
- type indicates whether the input corresponds to selecting a piece or a destination
- If return value is non-zero, output values are undefined

```
int getMenuSelection(int *selection);
```

Input:

- selection: Pointer to integer

Output:

- Returns 0 if selection is valid
- Returns non-zero if selection is invalid

Note:

- Used for menu navigation
 - The meaning of selection depends on the active menu context
-

input/command_parser.h**Responsibility**

Parses user textual input into structured command representations for further processing by the system.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [command.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int parseMoveCommand(const char *fromText, const
char *toText, Command *cmd);
```

Input:

- fromText: String representing the source position input by the user
- toText: String representing the destination position input by the user
- cmd: Pointer to a Command structure to store the parsed result

Output:

- Returns 0 if the command is successfully parsed
- Returns non-zero if parsing fails

Note:

- Converts user input from textual form into a structured Command object

- This function is part of the input processing pipeline and bridges user input and internal command representation
- The function validates the format of the input strings and translates them into internal position data
- If parsing fails, the Command structure may remain unchanged or contain partial data depending on implementation
- This function does not validate move legality and does not modify the game state

3.2.6. [UI Headers](#)

ui/main_menu.h

Responsibility

Handles the main menu interface and retrieves user selection.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - (none)

Data Type Defined in This File

- (none)

Function Signatures

```
int getMainMenuSelection(int *selection);
```

Input:

- selection: Pointer to store user selection

Output:

- Returns 0 if input succeeds
- Returns non-zero if input fails

Note:

- selection represents a menu option index
- If return value is non-zero, selection is undefined

ui/game_mode_menu.h

Responsibility

Handles selection of game mode (e.g., Human vs AI, Human vs Human).

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - (none)

Data Type Defined in This File

- (none)

Function Signatures

```
int getGameModeSelection(int *selection);
```

Input:

- selection: Pointer to store selected mode

Output:

- Returns 0 if selection succeeds
- Returns non-zero if selection fails

Note:

- selection maps to predefined game modes
-

ui/game_setup_menu.h

Responsibility

Collects configuration parameters for initializing a new game.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [game_config.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int getGameSetupConfig(GameConfig *config);
```

Input:

- config: Pointer to GameConfig

Output:

- Returns 0 if configuration succeeds
- Returns non-zero if input is invalid

Note:

- Populates all required configuration fields
 - If return value is non-zero, config is undefined
-

ui/endgame_menu.h**Responsibility**

Displays the end game menu as a modal overlay and collects user selection after game termination.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [game_state.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int showEndGameMenu(const GameState *state, int *selection);
```

Input:

- state: Pointer to the current game state
- selection: Pointer to an integer to store the user's menu selection

Output:

- Returns 0 if the menu is displayed and a selection is successfully made
- Returns non-zero if the operation fails

Note:

- Displays the end game menu as a modal interface layered above the gameplay UI

- The menu presents end-of-game information, including:
 - Game result (winner or draw)
 - Final game status
 - Total elapsed time
 - The function blocks until the user makes a selection
 - The selected option is stored in the provided selection pointer
 - Typical options may include:
 - Start a new game
 - Return to main menu
 - Exit application
 - This function does not modify the game state
-

ui/gameplay_ui.h

Responsibility

Provides rendering and display functions during gameplay.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int displayGameStatus(const GameState *state);
```

Input:

- state: Pointer to GameState

Output:

- Returns 0 if display succeeds
- Returns non-zero if display fails

Note:

- Displays information such as move count and game status

```
int displayTurn(Color turn);
```

Input:

- turn: Current player color

Output:

- Returns 0 if display succeeds
 - Returns non-zero if display fails
-

ui/dialog.h**Responsibility**

Handles confirmation dialogs that require user decision.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - (none)

Data Type Defined in This File

- (none)

Function Signatures

```
int confirmLeaveGame(int *result);
```

Input:

- result: Pointer to store user decision

Output:

- Returns 0 if input succeeds
- Returns non-zero if input fails

Note:

- result = 1 → confirm
- result = 0 → cancel
- If return value is non-zero, result is undefined

```
int confirmQuitGame(int *result);
```

Input:

- result: Pointer to store decision

Output:

- Returns 0 if input succeeds

- Returns non-zero if input fails

Note:

- result = 1 → confirm
 - result = 0 → cancel
 - If return value is non-zero, result is undefined
-

ui/error_popup.h**Responsibility**

Displays error messages to the user. Supports both non-fatal and fatal error reporting.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [error_code.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int showErrorMessage(ErrorCode code);
```

Input:

- code: Error code

Output:

- Returns 0 if display succeeds
- Returns non-zero if display fails

Note:

- Used for non-fatal errors
 - Message is transient and does not block execution
-

ui/board_renderer.h**Responsibility**

Collects configuration parameters for initializing a new game.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [game_config.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int renderBoard(const GameState *state);
```

Input:

- state: Pointer to current GameState

Output:

- Returns 0 if rendering succeeds
- Returns non-zero if rendering fails

Note:

- Does not modify GameState
 - If return value is non-zero, output may be incomplete
-

ui/fatal_error_window.h

Responsibility

Displays a fatal error window to inform the user of critical system failures and handles user acknowledgment before terminating or resetting the session.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [error_code.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int showFatalError(ErrorCode code);
```

Input:

- code: ErrorCode representing the fatal error condition

Output:

- Returns 0 if the fatal error window is displayed and acknowledged successfully
- Returns non-zero if the operation fails

Note:

- Displays a modal fatal error window with a user-friendly error message
 - The error message is obtained through the error mapping system (e.g., `getErrorMessage`)
 - This function blocks until the user acknowledges the error
 - Typical behavior includes:
 - Displaying the error message
 - Disabling all gameplay interactions
 - Waiting for user confirmation
 - After acknowledgment, control is returned to the system, which may transition to a safe state (e.g., main menu)
 - This function does not modify the game state directly
-

ui/game_setup_menu.h**Responsibility**

Collects configuration parameters for initializing a new game.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [game_config.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int getGameSetupConfig(GameConfig *config);
```

Input:

- config: Pointer to GameConfig

Output:

- Returns 0 if configuration succeeds
- Returns non-zero if input is invalid

Note:

- Populates all required configuration fields
 - If return value is non-zero, config is undefined
-

ui/move_input_widget.h**Responsibility**

Provides an interactive UI component for user move input and displays move format guidance.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [command.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int getWidgetMoveInput(Command *cmd);
```

Input:

- cmd: Pointer to a Command structure to store the user input

Output:

- Returns 0 if input is successfully captured and converted into a command
- Returns non-zero if input fails or is canceled

Note:

- Captures user move input through the move input widget
- This function handles user interaction for entering move data (e.g., source and destination positions)
- The captured input is translated into a structured Command object
- The function blocks until the user completes input or cancels the operation

- This function does not validate move legality and does not modify the game state

```
int showMoveFormatHint(void);
```

Input:

- None

Output:

- Returns 0 if the format hint is displayed successfully
- Returns non-zero if the operation fails

Note:

- Displays move format information to assist the user
- This function is typically triggered by an information icon within the move input widget
- The displayed content explains the expected format for move input
- The function does not modify any system state

3.2.7. [AI Headers](#)

ai/ai.h**Responsibility**

Generates moves for AI-controlled players and provides hint moves for user assistance.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)
 - [move.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int generateAIMove(const GameState *state, Move *move);
```

Input:

- state: Pointer to the current game state
- move: Pointer to a Move structure to store the generated move

Output:

- Returns 0 if a move is successfully generated
- Returns non-zero if move generation fails

Note:

- Generates a move for an AI-controlled player based on the current game state
- This function encapsulates all internal AI logic, including move evaluation and selection
- The implementation details (e.g., search strategy, heuristics, or optimizations) are not exposed through this interface
- The generated move is expected to be a valid and legal move
- This function does not modify the game state

```
int generateHintMove(const GameState *state, Move
*move);
```

Input:

- state: Pointer to the current game state
- move: Pointer to a Move structure to store the suggested move

Output:

- Returns 0 if a hint move is successfully generated
- Returns non-zero if hint generation fails

Note:

- Generates a recommended move for the current player to assist decision-making
- This function may reuse the same internal logic as AI move generation but does not execute the move
- It is intended for user-facing features such as hint systems
- This function does not modify the game state

3.2.8. [Services Headers](#)

log/log.h

Responsibility

Defines standardized error codes used across the system to represent non-fatal and fatal runtime errors.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [gamestate.h](#)
 - [move.h](#)
 - [gameconfig.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
int initLog(const GameConfig *config);
```

Input:

- config: Pointer to the game configuration

Output:

- Returns 0 if the log system is successfully initialized
- Returns non-zero if initialization fails

Note:

- initializes the logging system and prepares resources for recording game data
- Typically called once at the beginning of a new game session

```
int logGameStart(const GameConfig *config);
```

Input:

- config: Pointer to the game configuration

Output:

- Returns 0 if the game start is successfully logged
- Returns non-zero if logging fails

Note:

- Creates a new log entry for the start of a game
- Records initial configuration such as game mode and settings
- This function should be called when a new game begins

```
int logMove(const GameState *state, Move move);
```

Input:

- state: Pointer to the current game state
- move: The executed move to record

Output:

- Returns 0 if the move is successfully logged
- Returns non-zero if logging fails

Note:

- Appends a valid executed move to the log
- This function should only be called after a move has been validated and applied
- The logged data may include move details, turn information, and timestamps

```
int rebuildLogFromHistory(const GameState  
*state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if the log is successfully rebuilt
- Returns non-zero if rebuilding fails

Note:

- Reconstructs the move log from the current move history
- This function is typically invoked after an undo operation to ensure that the log remains consistent with the game state
- Existing log entries may be cleared and rebuilt based on the current MoveList

```
int logGameEnd(const GameState *state);
```

Input:

- state: Pointer to the current game state

Output:

- Returns 0 if the game termination is successfully logged
- Returns non-zero if logging fails

Note:

- Records the final result of the game and summary information
- The logged data includes:
 - GameResult (win, loss, draw, or termination)
 - Total elapsed time
- This function should be called when the game reaches a terminal state

```
void closeLog(void);
```

Input:

- None

Output:

- None

Note:

- Finalizes and releases any resources used by the logging system
- This function should be called at the end of a game session or when the application exits

[error/error.h](#)

Responsibility

Defines standardized error codes used across the system to represent non-fatal and fatal runtime errors.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - [error_code.h](#)

Data Type Defined in This File

- (none)

Function Signatures

```
const char *getErrorMessage(ErrorCode code);
```

Input:

- code: ErrorCode representing a specific error condition

Output:

- Returns a pointer to a constant string containing the corresponding error message

Note:

- Maps system-level error codes to user-facing messages
 - This function is used by UI components such as error popups and fatal error windows to display consistent messages to the user
 - The returned string is statically allocated and must not be modified by the caller
 - If the error code is unrecognized, a default fallback message may be returned
 - This function does not modify any system state
-

error/error_code.h**Responsibility**

Defines standardized error codes used across the system to represent non-fatal and fatal runtime errors.

Includes

- Standard Libraries:
 - (none required)
- Project Headers:
 - (none)

Data Type Defined in This File

- ErrorCode

```
typedef enum {  
    ERR_INVALID_INPUT,  
    ERR_EMPTY_SELECTION,  
    ERR_OPPONENT_PIECE,  
    ERR_ILLEGAL_MOVE,  
    ERR_UNRESOLVED_CHECK,  
    ERR_UNDO_UNAVAILABLE,  
    ERR_HINT_UNAVAILABLE,  
    ERR_NOT_YOUR_TURN,  
    ERR_TIME_UP,  
    ERR_FATAL  
    // Open to extension
```

```
} ErrorCode;
```

Description:

- The ErrorCode enumeration defines a unified set of error identifiers used throughout the system to handle invalid operations and exceptional conditions.
- These error codes enable consistent communication between system components, including:
 - Input handling modules
 - Gameplay validation and rule enforcement
 - AI and hint system
 - UI error display components

Usage:

- Returned or generated by validation, input, and control modules
- Translated into user-facing error messages by the UI layer
- Used to trigger specific system responses (e.g., blocking actions, displaying warnings, or terminating the game)

Note:

- These error codes represent non-fatal errors, unless explicitly specified (e.g., ERR_FATAL)
 - Error handling logic is separated from gameplay logic to maintain modularity
 - The mapping between ErrorCode and displayed messages is handled outside this module
 - Multiple modules may generate the same error code for consistency
-

3.3. Detailed Description of Input and Output Formats

This section defines the standardized I/O format for the program. Program input is provided by the user in all game states other than [INIT_STATE](#) and [EXIT_STATE](#), and keyboard input is used in user input parsing described in the [input processing layer](#). External program output is the log file described in the [log system](#).

3.3.1. The syntax or format of menu selections by the user

- **Input format:**
 - The user selects a menu option through the UI (mouse click or keyboard navigation). The selection is captured by UI input functions such as:

- getMenuSelection()
 - getGameModeSelection()
 - getGameSetupConfig()
- The UI may contain the following buttons to determine what has been selected (System State: Button 1, Button 2, ...):
 - Main Menu: Start Game, Quit Game
 - Game Mode Menu: Human vs Human, Human vs Computer, Computer vs Computer
- **Data processing:**
 - The UI module receives the user's selection through the corresponding input function
 - The selected value is validated against the allowed options for the current menu.
 - If valid:
 - The selection is used to update GameConfig (via getGameSetupConfig()) or converted into a control event, passed to the controller, enqueued into the ControlQueue, and processed by the FSM (processEvent())
 - Menu selections that trigger state transitions are internally represented as control events (e.g., EVENT_NEW_GAME, EVENT_EXIT_PROGRAM) and processed by the FSM.
 - The control layer enqueues the corresponding event into the appropriate queue, and the FSM processes the event to move the program to the next state
- **Effect on program:**
 - Menu input can directly affect the overall program state. This is different from ordinary gameplay input, because menu selections can move the system between major FSM states such as:
 - MAIN_MENU_STATE
 - GAME_MODE_SELECTION_STATE
 - GAME_SETUP_STATE
 - GAMEPLAY_STATE
 - END_GAME_MENU_STATE
 - EXIT_STATE

- Certain game flags are updated based on the options selected in the menu. Data in the timer system is updated when certain options in the [GAME_SETUP_STATE](#) are selected.
 - These updates mainly affect [GAMEPLAY_STATE](#), and will be reflected in the output log file.
- The effects include:
 - Starting a new game
 - User can select the game mode
 - Filling GameConfig
 - Enabling or disabling timers
 - Returning to the main menu
 - Exiting the program
- This is the only type of user input that can force the game state to update, for all other game state updates, the program will do so internally.

3.3.2. The syntax or format of a move entered by the user

- **Input format:**
 - Keyboard Input
 - There exists two text boxes provided by the [ui](#).
 - Text boxes will be labeled as “from” and “to” to indicate which data is being entered.
 - The user enters two board coordinates:
 - one for the source square (from)
 - one for the destination square (to)
 - Each coordinate should begin with:
 - one letter for the file or column
 - one number for the rank or row
 - Examples of possible coordinates:
 - A2
 - F4
 - g8
 - Rules:
 - Both from and to must be provided.
 - Letter case does not matter (ascii index 65-90 and 97-122 are both valid)
 - The coordinate must map to a valid board location defined in the [board representation section](#).
 - Extra trailing characters may be ignored by the parser, but the leading coordinate must be valid.

- Mouse Input:
 - Mouse Input: left click
 - Left click on a piece to move
 - Left click again on a destination square
 - One complete input cycle requires the user to perform both actions, the move validation pipeline will not begin until the cycle is completed.
- **Data processing:**
 - Keyboard Input:
 - Raw input is captured using `getMoveInput()` or `getBoardInput()`
 - Input strings are parsed into `Position` using `parseMoveCommand()`
 - A `Command` object is constructed using `createMoveCommand`
 - The move input is converted into an event, passed to the controller, enqueued into the `GameplayQueue`, and then processed by the FSM via `processEvent()`
 - Move validation is performed using:
 - `validateSelection()` (input-level validation)
 - `validateMove()` (rule-level validation)
 - If valid, the move is executed using `applyMove()`
 - Mouse Input:
 - Reads click positions from the UI
 - The clicked squares/tiles are converted directly into `Position` values
 - The two positions are combined into a move command or move-input event.
 - The event is passed to the controller, enqueued into the `GameplayQueue`, and processed through the same validation and execution pipeline used for keyboard input
- **Validation**
 - Input validation (format and bounds) in the input module
 - Gameplay validation (rules and legality) in `validateMove()`
- **Effect on program:**
 - If the move is valid:
 - The board state is updated via `applyMove()`
 - The move is appended to move history via `addMoveToHistory()`

- The turn is updated via switchTurn()
 - The move is recorded using logMove()
 - If the move ends the game, the FSM may transition toward termination handling
- If the move is invalid:
 - The game state is not changed
 - An error event (EVENT_ERROR) is generated using createErrorEvent(), passed to the controller, enqueued into the ControlQueue, and then processed by the FSM and handled by the UI layer
 - The player must enter a new move
-

3.3.3. The syntax or format of the log file

- **Output Location and File Management**

- The log file is a utility tool in the system services layer, the goal of the log file is for users to review previous games, and for developers to debug.
- The file will write to a txt file located in the logs/ directory, which will be named based on the time the log file is written.
- Entries of the file are in human readable language for future review.
- The log file is updated at importance points in execution:
 - When a game begins
 - When a valid move is added to moveHistory
 - When an undo event is triggered
 - When the game reaches [GAME_TERMINATION_STATE](#)

Log File Structure:

- **Header Section (Game Configuration)**

- The header records all configuration parameters defined during the setup phase
 - GAME_MODE_SELECTION_STATE
 - GAME_SETUP_STATE
- Contents include:
 - Game mode (e.g., Human vs Human, Human vs AI)
 - Player types (human or AI)
 - AI difficulty level (if applicable)
 - Timer settings

- Whether timer is enabled
- Initial time per player
- AI time limit

- **Move History Section**

- This section records all executed moves in chronological order. Each entry corresponds to a validated and successfully applied move.
- Entry Format:
 - [Move NN] HH:MM:SS | <Player> <Piece> <From> -> <To>
- Field Description:
 - Move Number (NN)
 - Sequential move index starting from 1
 - Zero-padded for alignment
 - Timestamp (HH:MM:SS)
 - Elapsed time since game start
 - Obtained from global clock
 - Player
 - Indicates the acting side: White or Black
 - If AI-controlled, explicitly labeled:
 - Example: Black (AI)
 - Piece
 - Type of piece moved (e.g. Pawn, Knight, Ant eater)
 - From / To
 - Board coordinates in standard format (e.g. F2, E4)

- **Termination Summary Section**

- This section is appended when the game reaches GAME_TERMINATION_STATE
- Contents:
 - Final Game result:
 - White Win / Black Win / Draw / Terminated by User
 - Termination reason:
 - Checkmate
 - Stalemate
 - Time Expiration
 - User Exit
 - Total elapsed time (from global clock)
 - Certain special moves will have extended entries which include details about the move.

4. Development Plan and Timeline

The development process follows an incremental, module-aligned approach, where each subsystem is implemented, tested, and integrated progressively.

Each phase corresponds to the architecture defined in [Section 1.2](#) and supports the control flow described in [Section 1.3](#).

Unit testing is performed alongside implementation to ensure correctness at every stage of development.

4.1. Partition of tasks

Phase A: Core Data Layer

Modules:

- core/

Files:

- include/position.h,
- include/piece.h,
- include/board.h,
- include/move.h,
- include/movelist.h
- include/gameconfig.h
- src/core/board.c,
- src/core/piece.c,
- src/core/move.c,
- src/core/position.c
- src/core/movelist.c
- src/core/gameconfig.c

Objectives:

- Define all core data structures (Board, Piece, Move, GameState)
- Implement board initialization and basic operations
- Implement position conversion utilities
- Implement shared helper functions and constants

Unit Tests:

- tests/test_board.c
- tests/test_piece.c
- tests/test_move.c

Validation:

- Board initializes correctly
- Pieces are placed correctly
- Position conversion is accurate

Phase B: Move Generation and Validation (Logic Core)**Modules:**

- gameplay/

Files:

- src/gameplay/rules_engine.c
- src/gameplay/move_validation.c

Objectives:

- Generate pseudo-legal moves
- Implement special moves (castling, en passant, etc.)
- Validate move legality

Unit Tests:

- tests/test_rules.c
- tests/test_movegen.c

Validation:

- All generated moves are correct
- Illegal moves are rejected
- Edge cases (check, capture, boundaries) handled

Phase C: Move Execution and State Management**Modules:**

- core/
- gameplay/

Files:

- gameplay/execution.c
- src/system/move_list.c
- src/system/game_state.c

Objectives:

- Execute moves and update GameState
- Implement undo functionality
- Maintain MoveList as the historical record for undo functionality
- Implement GameState initialization and update operations

Unit Tests:

- tests/test_move_execution.c
- tests/test_undo.c
- tests/test_game_state.c

Validation:

- Moves correctly update board state
- Undo restores exact previous state
- MoveList consistency is preserved

Phase D: Turn Management and Game Termination

Modules:

- turn/
- gameplay/

Files:

- src/turn/turn.c
- src/gameplay/endgame.c

Objectives:

- Implement turn switching
- Detect checkmate, stalemate, and end conditions

Unit Tests:

- tests/test_turn.c
- tests/test_endgame.c

Validation:

- Turn alternates correctly
- Game termination conditions are accurate

Phase E: Event System and Control Flow (Core Loop)

Modules:

- Main.c
- event/
- control/

Files:

- src/main.c
- src/event/event.c
- src/event/event_queue.c
- src/control/fsm.c
- src/control/controller.c

Objectives:

- Implement FSM state transitions ([Section 1.3.1](#))
- Implement Event and EventQueue data structures and operations
- Implement the unified move pipeline connecting input, validation, execution, and post-processing
- Implement gameplay loop skeleton
- Integrate event polling structure

Unit Tests:

- tests/test_control_flow.c (basic simulation)
- tests/test_event.c
- tests/test_fsm.c
- tests/test_controller.c

Validation:

- State transitions follow FSM definition
- Gameplay loop executes without UI
- Events are correctly enqueued and dequeued

Phase F: Input System

Modules:

- input/

Files:

- src/input/command.c
- src/input/input.c

Objectives:

- Parse user commands into Move objects
- Map input into Move Input Event

Unit Tests:

- tests/test_command.c
- tests/test_input.c

Validation:

- Valid commands parsed correctly
- Invalid commands rejected

Phase G: System Services (Log + Timer)

Modules:

- log/
- turn/
- time/

Files:

- src/log/log.c
- src/turn/turn_timer.c
- src/time/clock.c

Objectives:

- Implement persistent game logging (human-readable .log format)
- Implement turn timer with countdown and expiration detection
- Implement global clock for elapsed time tracking

Unit Tests:

- tests/test_log.c
- tests/test_timer.c
- tests/test_clock.c

Validation:

- Log file records moves in correct format
- Timer counts down accurately and triggers expiration event
- Log is rebuilt correctly after undo operations

Phase H: User Interface Layer**Modules:**

- ui/

Files:

- src/ui/main_menu.c
- src/ui/game_mode_menu.c
- src/ui/game_setup_menu.c
- src/ui/gameplay_ui.c
- src/ui/error_popup.c
- src/ui/fatal_error_window.c
- src/ui/endgame_menu.c
- src/ui/board_renderer.c
- src/ui/move_input_widget.c

Objectives:

- Implement Main Menu rendering and interaction
- Implement Game Mode Selection Menu and Game Setup Menu
- Implement Gameplay UI (board rendering, turn indicator, move history panel, move input widget, hint icon, clock, turn timer)
- Implement error feedback (Error Message Popup, Fatal Error Window)
- Implement End Game Menu (winner indicator, clock display, New Game / Back to Main Menu buttons)

- Integrate SDL2 rendering and event handling

Integration Tests:

- Menu navigation
 - Move interaction (mouse + command)
 - Error handling display
-

Phase I: AI Module**Modules:**

- ai/

Files:

- src/ai/ai.c

Objectives:

- Generate AI moves based on GameState
- Ensure compatibility with move pipeline

Unit Tests:

- tests/test_ai.c

Validation:

- AI generates only legal moves
 - AI integrates with gameplay loop
 - AI selects moves within time and depth constraints per difficulty
-

Phase J: Error System**Modules:**

- error/

Files:

- src/error/error.c

Objectives:

- Implement error code to message mapping
- Support consistent error handling across the system

Unit Tests:

- tests/test_error.c

Validation:

- All error codes map to correct messages
 - Unknown error codes return fallback message
-

Phase K: Full System Integration and Testing**Modules:**

- All modules

Objectives:

- Integrate all subsystems
- Perform end-to-end testing

System Tests:

- Full gameplay (Human vs Human, Human vs AI, AI vs AI)
 - Undo + log + timer interaction
 - Error handling and recovery
 - Game termination flow
-

4.2. Parallel Development Strategy

The modular architecture defined in [Section 1.2](#) enables **partial parallel development** once the core data structures are established.

After completion of Phase A (Core Data Layer), multiple subsystems can be developed concurrently due to clear interface boundaries and low coupling.

Phase dependency graph:

A → B → C → D → E → J → K
 ↘ F ↗
 ↘ H ↗
 ↘ G ↗

4.3. Development Timeline (Apr 12 – Apr 19)

The development schedule follows the dependency graph defined above, while leveraging parallel development whenever possible.

Apr 12:

- Complete Phase A (Core Data Layer)
- Finalize function APIs across all modules

Apr 13 – Apr 14 (Parallel Development Begins):

- Phase B (Logic: move generation & validation)
- Phase F (Input system: command parsing)
- Phase H (UI layer: basic menu structure)

Apr 15:

- Complete Phase B
- Begin Phase C (Move execution, MoveList, undo)
- Continue Phase F and Phase H

Apr 16:

- Complete Phase C
- Begin Phase D (Turn management, endgame detection)
- Begin Phase G (Timer module in parallel)

Apr 17:

- Complete Phase D
- Begin Phase E (Control flow skeleton, FSM loop)
- Continue UI integration (Phase H)

Apr 18:

- Complete Phase E
- Begin Phase I (AI module)
- Integrate input, UI, and system services

Apr 19:

- Phase J (Full system integration and testing)
- End-to-end gameplay validation
- Bug fixing and stabilization

4.4. Team member responsibilities

#	Task Name	Workload	Chief Person in charge	Deputy person in charge
A	Core Data Structures and Header Files	2	Yide	Yao
B	Move Generation and Validation	4	Yurang	Benjamin
C	Move Execution and State Management	3	Benjamin	Yurang
D	Turn Management and Game Termination	2	Benjamin	Yasith
E	Event System and Control Flow (Core Loop)	2	Yasith	Yide
F	Input System	2	Yide	Yasith
G	System Services (Log + Timer)	3	Yurang	Yao
H	User Interface Layer	4	Max	Yurang

I	AI Module	6	Yao	Max
J	Full System Integration and Testing	3	Max	Yide
/	/	31/Avg. ~5	/	/

5. Back Matter

5.1. Copyright

Software Title: Anteater Chess

Course: EECS 22L: Software Engineering Project in C, Spring 2026

Institution: University of California, Irvine

Development Team (Team 22: DeepAnteater):

Yao Li	yaol64@uci.edu
Yurang Li	yurangl@uci.edu
Yide Li	yidel3@uci.edu
Yasith Diunugala	ydiunuga@uci.edu
Max Zhang	maxyz@uci.edu
Benjamin Feng	benjaf3@uci.edu

Contact & Support: For software issues, please contact Team 22

5.2. References

- FIDE (Fédération Internationale des Échecs) Laws of Chess. Available at:
<https://www.fide.com/FIDE/handbook/LawsOfChess.pdf>
-

5.3. Index

Term	Page
------	------

GameState	4, 14, 34
Move	4, 12, 13, 27, 43
Board	3, 13, 25, 42
Event	3, 13, 29, 67
Command	3, 14, 28, 73
MoveList	4, 14, 33, 45
GameConfig	4, 13, 32, 49
Player	13, 31, 47
SystemState	4, 13, 32
Event Queue	3, 30, 68

Control Layer / FSM	4, 10, 71
Gameplay Logic	9, 54
AI	11, 86, 103
Log	4, 11, 87
Error System	11, 90, 91, 103